VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

$\sim \cdots * \cdots \sim$



Computer Engineering Project

Week 3 Progress Report

REAL-TIME PERSONAL PROTECTIVE EQUIPMENT (PPE) MONITORING SYSTEM USING EDGE AI AND IOTs

Instructor: **Dr. Vo Tuan Binh**

Name	ID
Pham Ngoc Mai Anh	2352063
Tran Quoc Hao	2352299
Nguyen Minh Khue	2352631

Ho Chi Minh City, 2026

Contents

1 Review of Previous Design Decisions **5**
1.1 Previous System Direction . . . 5
1.2 Issues Identified . . . 5
1.3 Current System Flow . . . 5

2 Experimental Comparison of YOLO11n and YOLO26n **7**
2.1 Training Configuration . . . 7
2.2 Validation Metrics and Quantitative Results . . . 8
2.3 Recorded Training Time . . . 9
2.4 Final Loss Values . . . 10
2.5 Training and Validation Curves . . . 10
2.6 Class-Specific Precision–Recall Analysis . . . 12
2.7 Normalized Confusion Matrix Analysis . . . 15
2.8 Qualitative Detection Results . . . 17
2.9 Discussion and Limitations . . . 19

3 Re-evaluation of Communication Protocols **22**
3.1 Actual Communication Requirements . . . 22
3.2 Communication Protocols . . . 22
3.3 Comparison . . . 29
3.3.1 Experimental Evidence from the Reviewed Studies . . . 30
3.3.2 Implications for the Proposed PPE Monitoring System . . . 33
3.3.3 Current Selection Direction . . . 33

4 Problem Definition and Deployment Requirements **35**
4.1 Project Context . . . 35
4.2 Deployment Challenges . . . 35
4.3 Evaluation Criteria . . . 35
4.4 Research Questions . . . 35

5 Review of Related Research **37**
5.1 Edge AI Background . . . 37
5.2 Optimization and Deployment Workflow . . . 37
5.3 Experimental Compression Study . . . 37
5.4 Cross-Paper Synthesis and Research Gaps . . . 38

6 Comparison of Model Optimization Methods **40**
6.1 Baseline and Comparison Approach . . . 40
6.2 Pruning . . . 40
6.2.1 Principle and Procedure . . . 40
6.2.2 Structured and Unstructured Approaches . . . 40
6.2.3 Evidence, Limitations, and Project Suitability . . . 40
6.3 Quantization . . . 40
6.3.1 Numerical Precision and Quantization Principle . . . 40
6.3.2 Post-Training Quantization and Quantization-Aware Training . . . 41
6.3.3 Static and Dynamic Quantization . . . 41
6.3.4 Calibration and Detection Accuracy . . . 41
6.3.5 Evidence, Limitations, and Project Suitability . . . 41
6.4 Knowledge Distillation . . . 41
6.4.1 Teacher–Student Approach . . . 41
6.4.2 Training Requirements and Project Suitability . . . 41
6.5 Combined Optimization Methods . . . 42
6.6 Comparative Assessment and Provisional Preference . . . 42

7 Deployment Tools, ONNX, and Computation Graphs **44**
7.1 From Trained Model to Deployment . . . 44
7.2 Training Frameworks . . . 44
7.2.1 PyTorch . . . 44
7.2.2 TensorFlow . . . 44
7.3 Candidate Inference Runtimes . . . 44
7.3.1 ONNX Runtime . . . 44

7.3.2 NVIDIA TensorRT 44
7.3.3 TensorFlow Lite 45
7.4 ONNX as an Intermediate Model Representation 45
7.5 Computation Graph Inspection Using Netron 45
7.5.1 Inspection Procedure 45
7.5.2 Graph Observations and Potential Compatibility Issues 45
7.6 Tool Comparison and Proposed Workflow 46

8 Hardware and Deployment Configuration 48
8.1 Device comparision 48
8.2 Interpretation of Published Benchmarks 50
8.3 Proposed Deployment Configuration 50
8.4 Planned Validation 50
8.4.1 Experimental Controls 50
8.4.2 Measurements and Reporting 51

9 Research Summary and Next Steps 51
9.1 Synthesis of Literature Findings 51
9.2 Provisional Selections and Justification 51
9.3 Comparison with Reviewed Studies 51
9.4 Remaining Research and Planned Experiments 52

10 Revised System Architecture 53
10.1 Architecture Revision Rationale 53
10.2 Revised End-to-End System Flow 53
10.3 Current Provisional Configuration 54

11 Evaluation Plan 55
11.1 Model Evaluation 55
11.2 Edge Deployment Evaluation 55
11.3 Communication Protocol Evaluation 55
11.4 System-Level Evaluation 55

11.5 Experimental Workflow 56

12 Week 3 Findings and Next Steps **56**

12.1 Current Findings 56

12.2 Remaining Decisions 57

12.3 Next Steps 57

1 Review of Previous Design Decisions

1.1 Previous System Direction

In the previous stages, the project was formulated as an Edge AI-based PPE monitoring system for construction environments. The overall processing flow consists of video acquisition from cameras, local inference on an edge device, transmission of violation events through a communication layer, and visualization or storage at a centralized backend or dashboard.

YOLO-based object detection models were selected as the main detection direction due to their suitability for real-time inference. For the communication layer, MQTT was initially proposed as the telemetry mechanism for transmitting lightweight JSON-based PPE violation alerts from the edge device to the monitoring system.

At the same time, the Week 1 report also acknowledged that edge-to-cloud communication may rely on different IoT protocols, including MQTT, CoAP, and HTTP/REST, depending on the application requirements.

1.2 Issues Identified

After reviewing the previous design decisions, two main issues were identified and require further investigation before the system architecture is finalized.

- The edge hardware and deployment environment have not yet been sufficiently benchmarked. Quantitative measurements such as inference latency, FPS, memory usage, and computational load are still needed to evaluate whether the selected platform can satisfy the real-time requirements of the PPE monitoring system.
- The communication layer was previously oriented toward MQTT at an early stage. However, other candidate protocols such as HTTP and CoAP have not yet been studied and compared at the same level. Therefore, the group needs to investigate and compare different communication approaches before selecting the protocol that best matches the system requirements and actual workload.

Although the previous report presented MQTT as the preferred communication approach, this represented an early design direction rather than a final decision supported by comparative experimental evaluation. Therefore, Week 3 revisits this assumption by evaluating HTTP, MQTT, and CoAP under the same system requirements.

1.3 Current System Flow

The current architecture consists of a camera input, an Edge AI processing pipeline, a generic communication layer, a backend server, and a monitoring dashboard.

The AI pipeline generates PPE violation events and stores them in a Violation/Result Queue. These events are then forwarded through the Communication Layer to the Backend/Server. At this stage, the communication protocol has not yet been finalized.

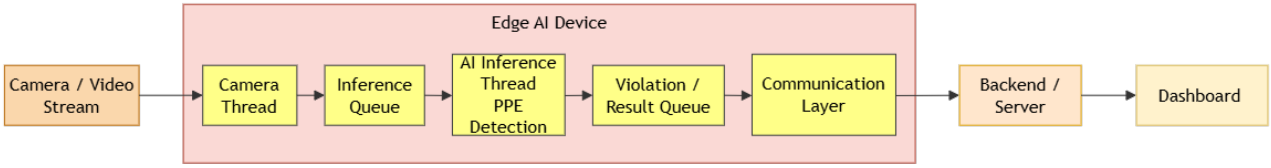


Figure 1: Current system architecture for the Edge AI-based PPE monitoring system.

The local PPE detection pipeline should continue operating independently of network availability. Temporary network interruptions should affect only the transmission of results, not the AI inference process.

2 Experimental Comparison of YOLO11n and YOLO26n

YOLO11n and YOLO26n were fine-tuned for construction-site personal protective equipment (PPE) detection. Both experiments were configured for 30 epochs using the same dataset configuration and nominal training settings. This section compares their validation accuracy, recorded training duration, loss curves, class-specific performance, and qualitative detection results.

The experimental information was extracted from the `args.yaml`, `results.csv`, and visualization files included in the two training archives.

2.1 Training Configuration

Both models were initialized from pretrained checkpoints and trained using the Metal Performance Shaders (MPS) backend. The dataset configuration was `construction-ppe.yaml`. Validation was enabled during training using the validation split.

Table 1 summarizes the shared configuration.

Table 1: Shared training configuration for YOLO11n and YOLO26n.

Setting	**Configured value**
Task	Object detection
Training epochs	30
Batch size	4
Input image size	640×640
Device backend	MPS
Pretrained initialization	Enabled
Optimizer selection	Automatic
Initial learning-rate setting	0.01
Final learning-rate factor	0.01
Momentum setting	0.937
Weight decay	0.0005
Warm-up epochs	3
Early-stopping patience	100
Random seed	0
Deterministic mode	Enabled
Data fraction	1.0
Data-loading workers	0
Nominal batch size	64

The optimizer was configured as `auto`. Therefore, the learning-rate and momentum entries in Table 1 describe the saved configuration and should not be interpreted as independently

verified values selected by the automatic optimizer.

The actual training batch size was 4. The nominal batch size of 64 is a separate reference setting used by the training implementation.

The main augmentation settings were identical. Mosaic augmentation was configured with probability 1.0 and disabled for the final 10 epochs. Horizontal flipping used probability 0.5, translation used a magnitude of 0.1, and scaling used a magnitude of 0.5. The HSV augmentation settings were 0.015, 0.7, and 0.4 for hue, saturation, and value, respectively. MixUp and CutMix were disabled.

These shared settings establish a comparable experimental framework. However, automatic optimizer selection does not guarantee that both models used identical optimization behavior.

2.2 Validation Metrics and Quantitative Results

The models were evaluated using precision, recall, mAP_{50}, and $\text{mAP}_{50:95}$.

Precision measures the proportion of predicted detections that are correct, whereas recall measures the proportion of annotated objects that are successfully detected:

$$\text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \frac{TP}{TP + FN}, \tag{1}$$

where TP, FP, and FN denote true positives, false positives, and false negatives, respectively, under the evaluation matching criteria.

The metric mAP_{50} evaluates average precision at an intersection-over-union (IoU) threshold of 0.50. The stricter metric $\text{mAP}_{50:95}$ averages performance over IoU thresholds from 0.50 to 0.95 in increments of 0.05.

Results at the selected epoch. For Table 2, the epoch with the highest recorded validation $\text{mAP}_{50:95}$ was selected for each model. Both models achieved this maximum at epoch 28. All four metrics in each column were taken from that same epoch; the table does not combine independently selected metric maxima.

Table 2: Validation results at the epoch with the highest recorded $\text{mAP}_{50:95}$.

Metric	**YOLO11n**	**YOLO26n**
Selected epoch	28	28
Precision (%)	71.137	64.295
Recall (%)	53.911	50.772
mAP_{50} (%)	57.624	54.334
$\text{mAP}_{50:95}$ (%)	28.048	27.098

At the selected epoch, YOLO11n exceeded YOLO26n by 6.842 percentage points in precision, 3.139 percentage points in recall, 3.290 percentage points in mAP_{50}, and 0.950 percentage points in $\text{mAP}_{50:95}$.

These results indicate higher aggregate validation performance for YOLO11n within the recorded 30-epoch training budget.

Results at the final epoch. Table 3 reports the validation results after the full 30 epochs.

Table 3: Validation results at epoch 30.

Metric	**YOLO11n**	**YOLO26n**
Precision (%)	65.968	59.746
Recall (%)	55.028	52.389
mAP_{50} (%)	57.130	53.802
$\text{mAP}_{50:95}$ (%)	28.003	26.887

Between epoch 28 and epoch 30, recall increased for both models, while precision and mAP decreased. This illustrates that the metrics do not necessarily improve together: a model may recover more objects while producing a less favorable overall precision–recall trade-off.

The reductions in $\text{mAP}_{50:95}$ were small: 0.045 percentage points for YOLO11n and 0.211 percentage points for YOLO26n. These changes over two epochs alone are insufficient to establish overfitting.

2.3 Recorded Training Time

The cumulative time recorded at epoch 30 was 1,477.04 seconds for YOLO11n and 2,076.08 seconds for YOLO26n. Table 4 summarizes these durations.

Table 4: Recorded duration of the 30-epoch training runs.

Measurement	**YOLO11n**	**YOLO26n**
Total recorded time (s)	1477.04	2076.08
Total recorded time (min)	24.62	34.60
Average time per epoch (s)	49.23	69.20

The average epoch duration was calculated by dividing the final cumulative time by 30. It is an average of the recorded run duration, rather than an isolated measurement of a single training step.

YOLO26n required 599.04 additional seconds, corresponding to approximately 40.56% more recorded training time:

$$\frac{2076.08 - 1477.04}{1477.04} \times 100 \approx 40.56\%. \tag{2}$$

These measurements describe training duration, not inference latency. They cannot establish which model processes new images faster. Furthermore, a single run per model does not quantify runtime variability.

2.4 Final Loss Values

Table 5 presents the loss values recorded at epoch 30. YOLO11n reports a Distribution Focal Loss (DFL) component, whereas YOLO26n reports an L1 component.

Table 5: Loss values recorded at epoch 30. A dash denotes a component absent from that model's CSV.

Loss component	**YOLO11n**	**YOLO26n**
Training box loss	1.44712	1.53681
Training classification loss	1.03987	1.34623
Training DFL loss	1.54896	—
Training L1 loss	—	0.01578
Validation box loss	1.69968	1.65970
Validation classification loss	1.15957	1.49425
Validation DFL loss	1.72818	—
Validation L1 loss	—	0.01796

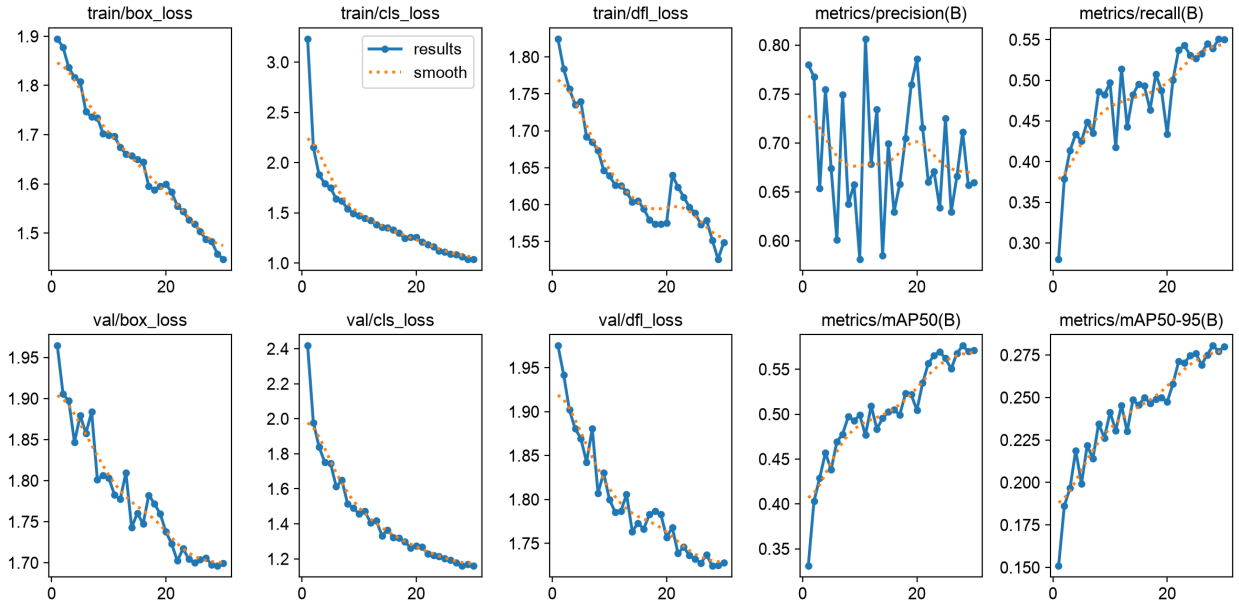
Different bounding-box objectives. YOLO11n uses DFL to supervise distribution-based bounding-box regression. YOLO26n removes DFL and reports an L1 loss on normalized bounding-box distances. Consequently, the missing entries indicate different loss formulations rather than missing experimental data.

The DFL value of 1.54896 and the L1 value of 0.01578 cannot be compared directly because they represent different objectives and numerical scales. Loss trends are useful for monitoring each model's optimization, while precision, recall, and mAP provide a more appropriate basis for comparing detection quality.

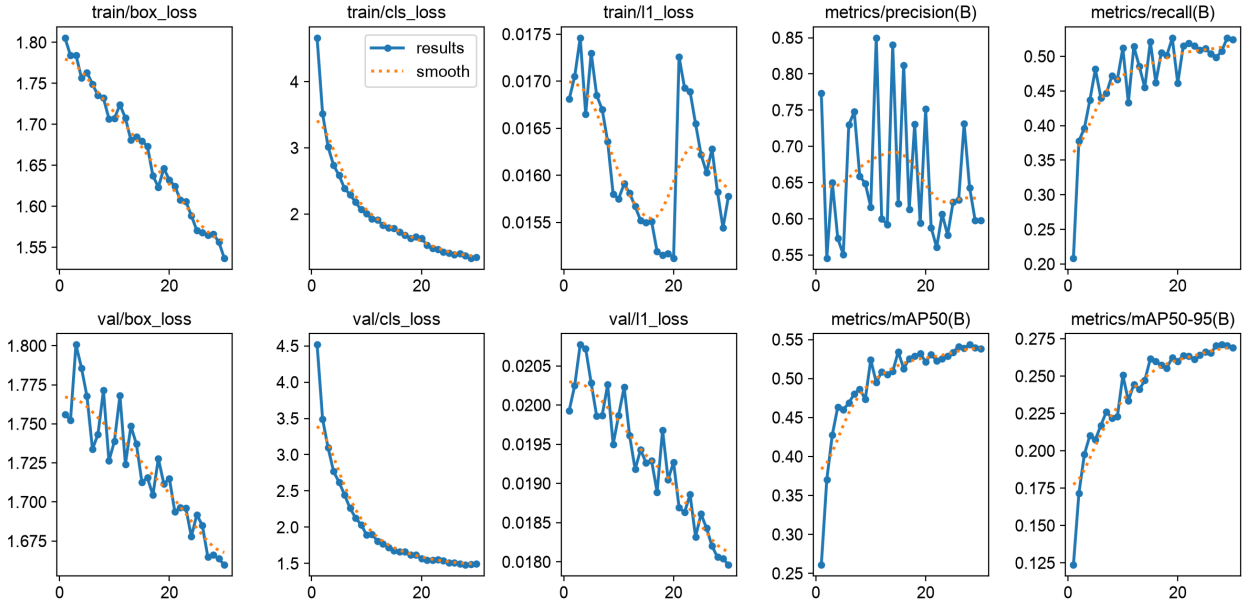
2.5 Training and Validation Curves

Figure 2 presents the training and validation curves recorded over 30 epochs. The panels retain their original axis scales, so apparent curve heights should not be compared without inspecting the numerical axes.

YOLO11n training behavior. For YOLO11n, training box loss decreased from 1.89470 at epoch 1 to 1.44712 at epoch 30. Classification loss decreased from 3.23225 to 1.03987, and DFL



(a) YOLO11n training and validation curves.



(b) YOLO26n training and validation curves.

Figure 2: Training progress of YOLO11n and YOLO26n over 30 epochs.

decreased from 1.82448 to 1.54896. Although individual components fluctuated, their overall trends show that the training objectives were increasingly optimized.

Validation losses also decreased overall. Validation box loss fell from 1.96435 to 1.69968, classification loss decreased from 2.41694 to 1.15957, and DFL decreased from 1.97509 to 1.72818. Meanwhile, validation $\text{mAP}_{50:95}$ increased from 15.087% to 28.003%.

The simultaneous reduction in validation losses and improvement in validation mAP provides evidence of improved performance on the validation split.

YOLO26n training behavior. For YOLO26n, training box loss decreased from 1.80549 to 1.53681, and classification loss decreased from 4.66044 to 1.34623. The L1 component fluctuated and changed from 0.01681 to 0.01578.

Validation box loss decreased from 1.75601 to 1.65970, classification loss decreased from 4.51593 to 1.49425, and L1 loss decreased from 0.01993 to 0.01796. Validation $\text{mAP}_{50:95}$ increased from 12.359% to 26.887%.

Thus, YOLO26n also learned during the experiment, despite achieving lower aggregate validation accuracy than YOLO11n.

Precision fluctuations. Precision varied considerably between epochs for both models, whereas recall followed a clearer overall upward trend. The curves therefore do not support a claim that every aspect of detection quality improved monotonically.

Precision and recall describe different aspects of detection. Their fluctuations should be interpreted alongside mAP, rather than using a single high-precision epoch as evidence of the best overall model.

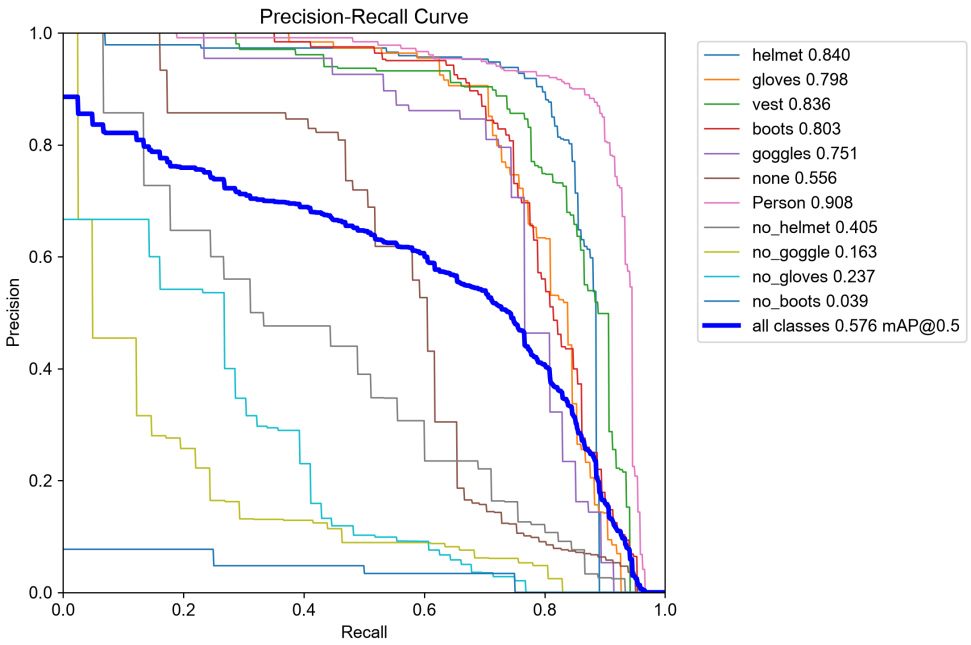
Selection of epoch 28. The highest recorded $\text{mAP}_{50:95}$ occurred at epoch 28 for both models. This is why epoch 28 was selected for the best-recorded-results table. It is a reporting criterion applied to the CSV measurements, rather than a predetermined epoch.

Both training and validation curves show overall improvement. The limited late-epoch changes do not provide sufficient evidence to diagnose overfitting or establish that either model had fully converged.

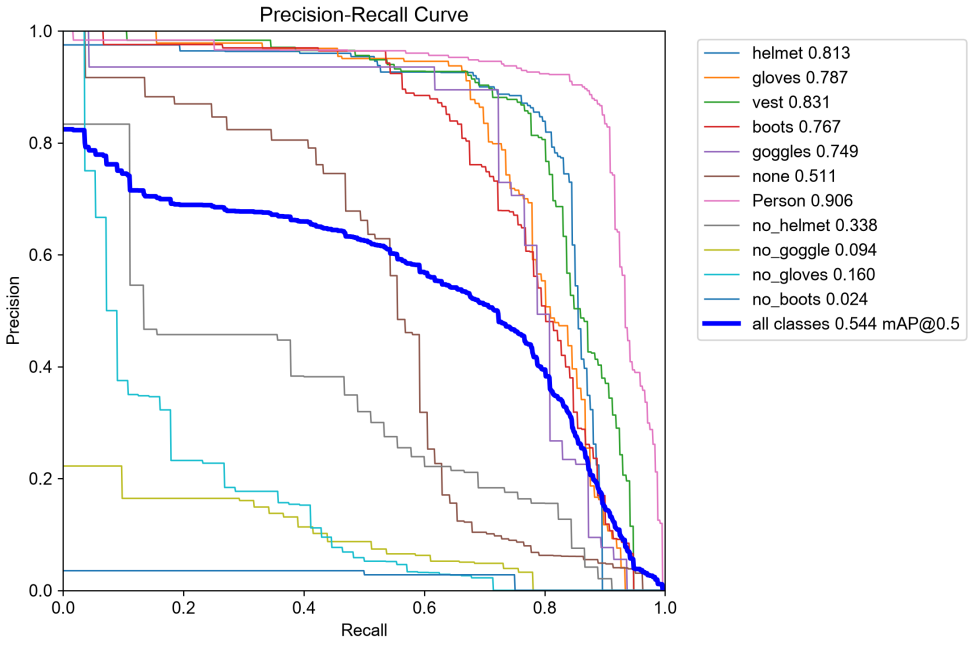
2.6 Class-Specific Precision–Recall Analysis

Figure 3 presents the saved precision–recall curves. Each curve describes the trade-off between precision and recall as the confidence threshold varies.

The class-specific values in the legends are AP50 values, rather than precision measurements at a single threshold.



(a) YOLO11n precision–recall curves.



(b) YOLO26n precision–recall curves.

Figure 3: Precision–recall curves from the saved validation outputs.

Table 6 reproduces the values shown in the curve legends. These values are rounded to three decimal places in the original figures and are reported separately from the higher-precision epoch measurements in the CSV.

Table 6: Class-specific AP50 read from the saved precision–recall curve legends.

Class	YOLO11n AP50	YOLO26n AP50
`helmet`	0.840	0.813
`gloves`	0.798	0.787
`vest`	0.836	0.831
`boots`	0.803	0.767
`goggles`	0.751	0.749
`none`	0.556	0.511
`Person`	0.908	0.906
`no_helmet`	0.405	0.338
`no_goggle`	0.163	0.094
`no_gloves`	0.237	0.160
`no_boots`	0.039	0.024

Person and visible PPE detection. The `Person` class achieved the highest AP50 for both models: 0.908 for YOLO11n and 0.906 for YOLO26n. Helmet, vest, gloves, and boots also achieved substantially higher AP50 than the corresponding missing-equipment classes where available.

The curves indicate that detecting people and visible protective equipment was more successful than detecting equipment absence in these experiments. YOLO11n had higher displayed AP50 for every class, although the differences for `Person`, `goggles`, and `vest` were small.

Missing-equipment detection. Both models performed poorly on several missing-PPE categories. For `no_goggle`, AP50 was 0.163 for YOLO11n and 0.094 for YOLO26n. For `no_gloves`, the values were 0.237 and 0.160. The lowest results occurred for `no_boots`, with AP50 values of only 0.039 and 0.024.

These weaknesses are important for safety monitoring. A model may correctly detect a worker without correctly identifying the worker's missing equipment. Consequently, high person-detection performance should not be interpreted as reliable PPE-violation detection.

Possible causes requiring further investigation. The curves identify performance differences but do not establish their causes. Class frequency, annotation consistency, object visibility, and image diversity should be examined before attributing the errors to a particular architectural feature.

Additional data or longer training may help, but their benefits must be demonstrated through further experiments.

2.7 Normalized Confusion Matrix Analysis

Figure 4 presents the normalized confusion matrices. The horizontal axis contains the true classes, and the vertical axis contains the predicted classes.

The displayed matrices are normalized by true-class columns. A diagonal entry therefore represents the fraction of objects in that column assigned to the correct class under the matrix's evaluation settings.

Background entries and the annotated none class. The bottom background row represents unmatched ground-truth objects, while the rightmost background column represents unmatched predictions.

The dataset class `none` is a separate annotated category. It must not be confused with background, which is used by the evaluation matrix to represent unmatched objects or predictions.

Correct class assignments. The `Person` diagonal entry is approximately 0.90 for both models. Helmet entries are approximately 0.83 for YOLO11n and 0.85 for YOLO26n, while vest entries are approximately 0.81 for both.

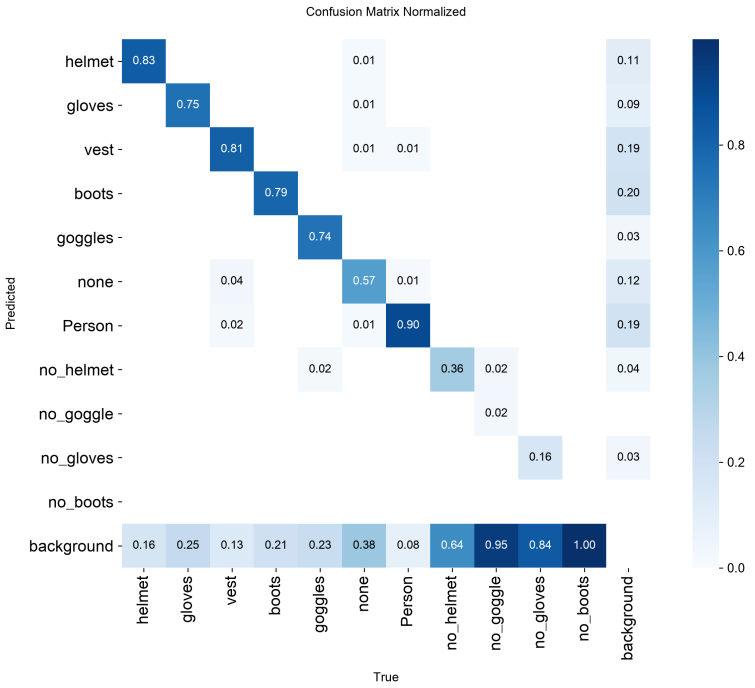
YOLO26n's higher helmet diagonal entry does not contradict its lower helmet AP50. The confusion matrix describes performance under particular matching and confidence settings, whereas AP50 summarizes the precision–recall curve across thresholds.

Thus, a better diagonal entry for one class does not necessarily imply a better AP50 for that class.

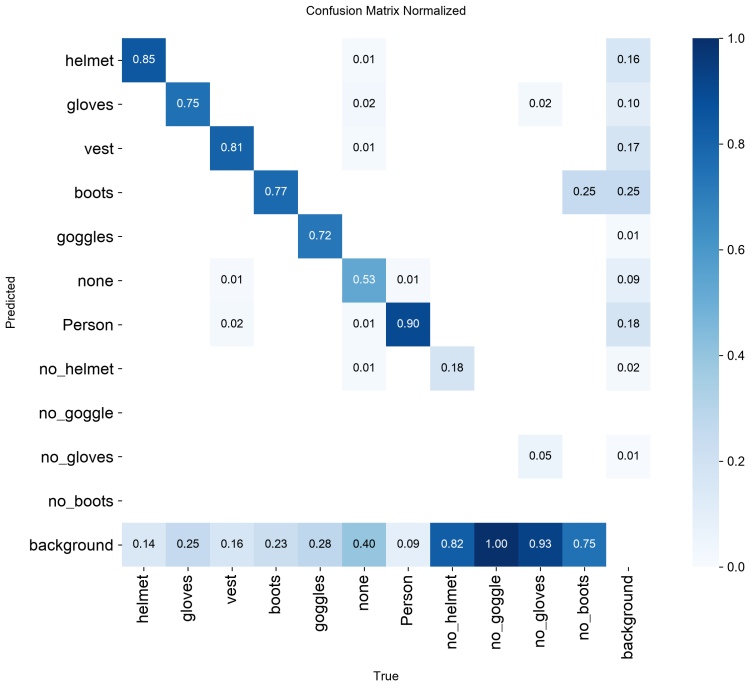
Missed missing-PPE objects. The background row reveals substantial missed detections for missing-PPE classes. For true `no_helmet` objects, the displayed unmatched proportions are approximately 64% for YOLO11n and 82% for YOLO26n.

For `no_gloves`, the corresponding proportions are approximately 84% and 93%. For `no_goggle`, approximately 95% are unmatched for YOLO11n, while all are unmatched for YOLO26n under the matrix's settings.

These results are consistent with the low AP50 values for missing-equipment categories. They indicate that missed objects are an important source of error, rather than only confusion between correctly localized objects.



(a) YOLO11n normalized confusion matrix.



(b) YOLO26n normalized confusion matrix.

Figure 4: Comparison of normalized confusion matrices.

Small evaluation support for no-boots. The raw YOLO26n confusion matrix contains only four ground-truth `no_boots` instances. Three are unmatched, while one is assigned to `boots`. Therefore, the displayed unmatched proportion of 0.75 is based on very few examples.

This should not be interpreted as reliable detection: the correct-class diagonal is zero, and the displayed AP50 is only 0.024. The small number of examples also limits the stability of conclusions for this category.

Interpretation of unmatched predictions. The normalized background column shows how unmatched predictions are distributed across predicted classes. Its entries are not individual class false-positive rates.

For direct comparisons of false-detection counts, the raw `confusion_matrix.png` files should be examined alongside the normalized matrices.

2.8 Qualitative Detection Results

Figures 6–8 present the prediction mosaics saved for the validation batches. They illustrate bounding-box placement, predicted classes, confidence scores, and representative detection errors.

The batch-0 prediction mosaics contain corresponding visible filenames. They can be inspected against the ground-truth mosaic shown in Figure 5.

Person detection without complete PPE assessment. In `image1397.jpg`, both models display detections for `Person`, `none`, and `no_helmet`. However, the ground-truth mosaic also contains `no_gloves` annotations that are absent from both displayed prediction panels.

This example shows that detecting a person and one missing item does not guarantee that all annotated PPE conditions are recognized.

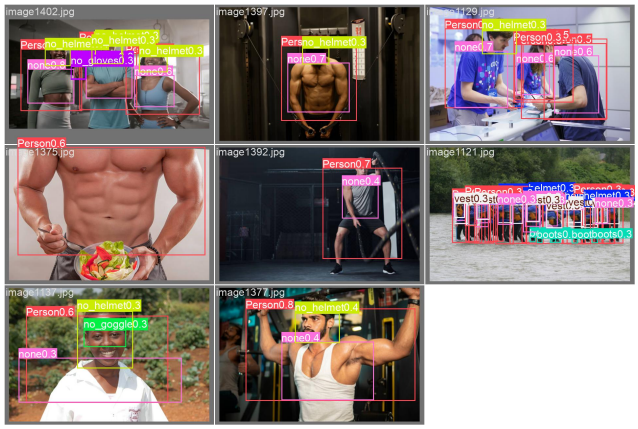
Differences in missing-PPE predictions. In `image1137.jpg`, YOLO11n displays predictions for `no_helmet` and `no_goggle`, alongside `Person` and `none`. YOLO26n displays `Person` and `none`, but does not show the two missing-PPE detections.

This example is consistent with YOLO11n's higher AP50 for these categories. However, a single image cannot establish the overall frequency or magnitude of the difference.

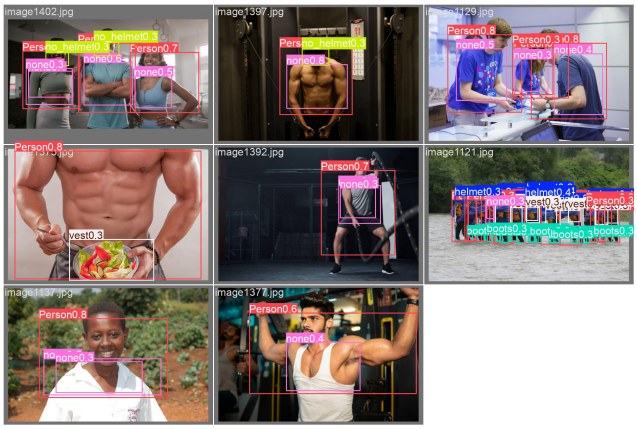
An incorrect equipment prediction. In `image1375.jpg`, YOLO26n displays a `vest` prediction, with confidence rounded to 0.3, over the bowl and lower-torso region. The corresponding ground-truth panel contains no vest annotation there.



Figure 5: Ground-truth annotations for validation batch 0, used as a reference for the qualitative comparison.



(a) YOLO11n predictions: validation batch 0.



(b) YOLO26n predictions: validation batch 0.

Figure 6: Saved prediction mosaics for validation batch 0.

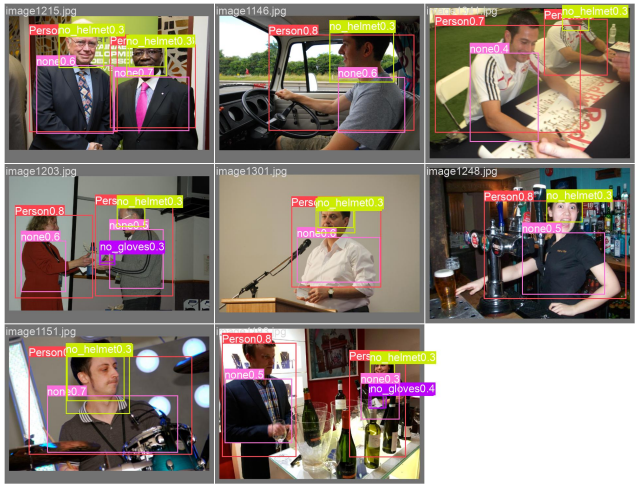
Both models also omit the annotated `no_gloves` objects in this image. YOLO11n avoids the displayed vest error, but still does not recover all annotated objects.

This demonstrates why qualitative assessment should consider both incorrect predictions and missed detections.

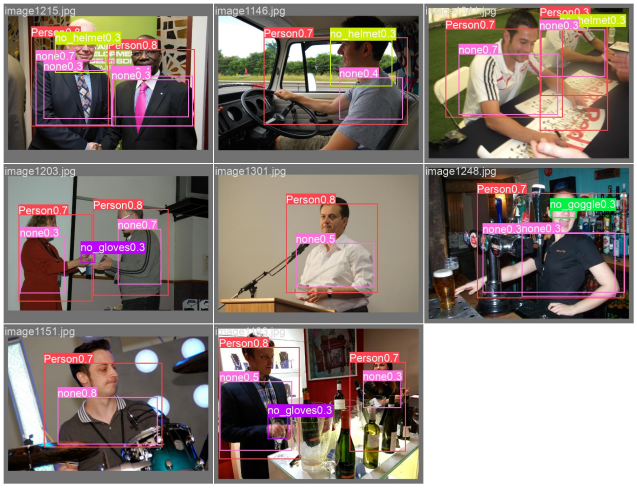
Crowded scenes and confidence scores. The prediction panels for `image1121.jpg` contain many overlapping boxes and labels. Larger individual-image views would make localization and overlapping predictions easier to inspect than the batch mosaics.

Several displayed confidence scores are rounded to approximately 0.3. A higher score on an individual prediction does not establish better model accuracy or calibration. Correctness must be assessed against annotations, and overall detection quality must be evaluated using quantitative metrics.

Additional validation examples. Figures 7 and 8 provide additional visual examples. The saved images are illustrative outputs; they do not provide a complete error count or confirm identical checkpoint selection and prediction thresholds.



(a) YOLO11n predictions: validation batch 1.



(b) YOLO26n predictions: validation batch 1.

Figure 7: Saved prediction mosaics for validation batch 1.

2.9 Discussion and Limitations

Comparison under the current training budget. Under the recorded 30-epoch configuration, YOLO11n achieved higher aggregate validation precision, recall, and mAP than YOLO26n at both the selected epoch and the final epoch. It also had higher displayed class-specific AP50 and completed the training run in less recorded time.



(a) YOLO11n predictions: validation batch 2. (b) YOLO26n predictions: validation batch 2.

Figure 8: Saved prediction mosaics for validation batch 2.

These findings support YOLO11n as the stronger validation baseline for the current experiment. They do not establish that it will outperform YOLO26n under every training budget, optimization strategy, or deployment environment.

Implications for PPE monitoring. Both models performed substantially better on person and visible-equipment detection than on several missing-PPE categories. This distinction is important for safety monitoring: a correctly detected worker may still have an undetected equipment violation.

Therefore, aggregate mAP should be supplemented with class-specific results, missed-detection analysis, and representative failure cases. The low performance on `no_goggle`, `no_gloves`, and `no_boots` is a significant limitation of the current experiments.

Experimental scope. The comparison contains one training run per model using a single random seed and automatic optimizer selection. It does not quantify variation across repeated runs.

The evaluation support for some classes is small, especially `no_boots`. Additional annotated evaluation examples are needed to support more stable conclusions for these categories.

The saved visualizations and per-epoch CSV measurements are reported as their respective outputs. They should not be treated as proof that every figure was generated from the same

checkpoint and prediction settings without checking the original evaluation procedure.

Inference performance. The archives do not contain inference latency measurements. Consequently, the shorter recorded training duration of YOLO11n cannot establish faster prediction.

A separate benchmark using identical hardware, input resolution, batch size, numerical precision, and execution backend is required to compare inference latency and deployment throughput.

Further evaluation. Future experiments should audit annotation consistency and class coverage, particularly for missing-PPE categories. Additional data, revised training settings, or longer training should be assessed through controlled experiments rather than assumed to improve performance.

Finally, the reported accuracy measurements are validation results used during model development. Evaluation on an independent test set is required to assess final generalization performance.

3 Re-evaluation of Communication Protocols

3.1 Actual Communication Requirements

During the development of the proposed PPE monitoring system, the group identified that the communication layer must satisfy several practical requirements rather than simply provide a connection between the Edge AI device and the backend.

First, the system should maintain reliable data transmission under unstable network conditions. Since construction-site connectivity may experience packet loss or temporary disconnection, unsent violation events should be recoverable through appropriate buffering, retransmission, or reconnection mechanisms.

Second, when a PPE violation is detected, the corresponding event should be delivered to the backend with sufficiently low latency so that the dashboard or responsible manager can receive the update in a timely manner.

Third, communication failures should not interrupt the local PPE detection process. The Edge AI device should continue performing camera acquisition and inference independently, while the communication layer handles network failures, reconnection, and retransmission in the background.

During the implementation and literature review process, the group also recognized that different communication protocols may satisfy these requirements in different ways. Instead, several candidate protocols should be evaluated according to the actual requirements and network conditions of the proposed system.

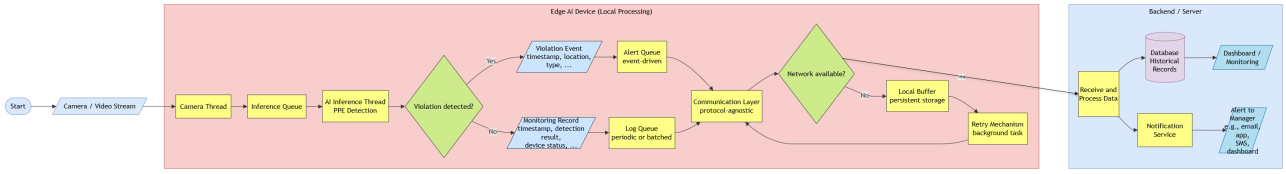


Figure 9: Current protocol-agnostic system flow for the Edge AI-based PPE monitoring system.

3.2 Communication Protocols

The communication layer plays an important role in connecting the Edge AI device with the backend system, particularly for transmitting violation alerts and monitoring data. Since the proposed PPE monitoring system is intended for construction-site environments, where network connectivity may be unstable, the communication mechanism should remain reliable without affecting the local PPE detection process.

In the previous design, MQTT was initially considered as the main communication protocol. However, after reviewing several research studies and external technical references, the

group found that HTTP, MQTT, and CoAP provide different advantages depending on network conditions, resource constraints, reliability requirements, and backend integration needs. Therefore, the communication layer is revisited at this stage before a final protocol is selected.

The objective of this investigation is to understand the characteristics and trade-offs of each candidate protocol and to determine which approach is most suitable for the actual communication requirements of the proposed system.

Research 1

- **Title:** Benchmarking Buffer Size in IoT Devices Deploying REST HTTP
- **Author:** Cao Vien Phung, Mounir Bensalem and Admela Jukan
- **Year:** 2021
- **Link:** https://arxiv.org/pdf/2101.02615
- **Summary content:**

This study identified MQTT, CoAP, and REST HTTP as common application-layer communication protocols for IoT systems, while focusing their study specifically on REST HTTP due to its compatibility with existing software and network infrastructure. In their model, HTTP follows the conventional client–server request/response mechanism.

The main objective of the study was to investigate the behavior of the REST HTTP client buffer under lossy and intermittent network conditions. When a communication failure or timeout occurs, pending requests remain in the client buffer and are retransmitted after the connection becomes available again.

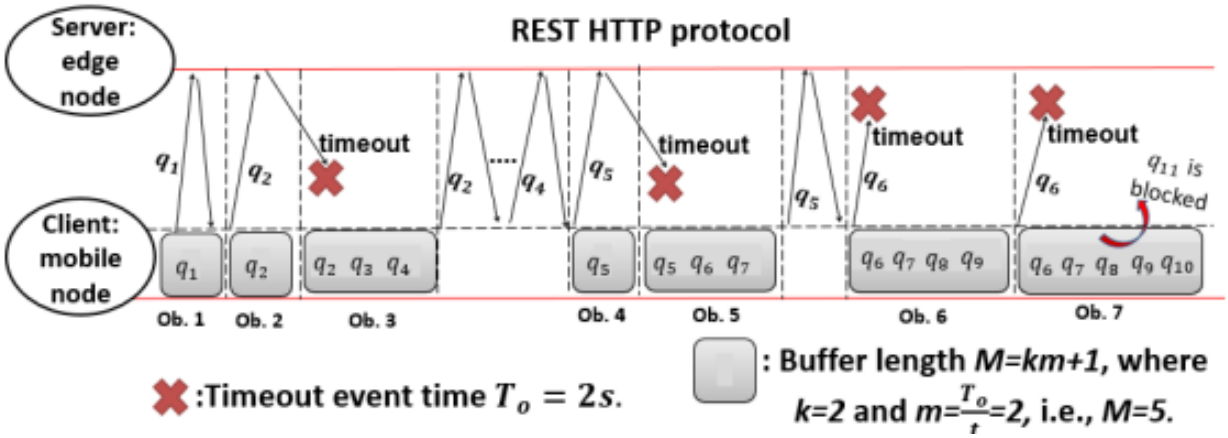


Figure 10: REST HTTP communication scenario under lossy network conditions.

The experimental results showed that increasing the client buffer size does not always improve communication performance in lossy environments. Under certain loss conditions, a larger buffer may lead to more accumulated or blocked messages. Therefore, the buffer size should be selected according to actual network conditions rather than simply being maximized.

This study is relevant to the proposed PPE monitoring system because network connectivity at construction sites may be unstable. If REST HTTP is considered for the communication layer, local buffering, timeout handling, and retransmission should be carefully designed, while ensuring that temporary communication failures do not interrupt the local AI inference process.

Research 2

- **Title:** Empirical Evaluation of MQTT, CoAP and HTTP for Smart City IoT Applications
- **Author:** Rafid M. Khaleefah, Ali A. Abed, Mahmood A. Al-Shareeda

- **Year:** 2025
- **Link:** https://www.researchgate.net/publication/395833265$_{E}mpirical_{E}valuation_{o}f_{M}QTT_{C}oAP_{a}nd$

- **Summary content:**

The study conducted an empirical evaluation of three widely used IoT communication protocols, including MQTT, CoAP, and HTTP, using a Raspberry Pi-based physical testbed. The motivation of the study was to investigate the trade-offs among different communication approaches rather than assuming that a single protocol is suitable for every IoT application.

The three protocols were evaluated under identical conditions using Raspberry Pi 4 devices, the same payload size, and the same transmission interval. MQTT, CoAP, and HTTP were implemented using Mosquitto, libcoap, and Flask, respectively, enabling a fair direct comparison. [2].

Four performance metrics were considered in the experiment:

- **Latency**: the time between message transmission and reception of the corresponding acknowledgment.
- **Message overhead**: the total amount of transmitted data, including both the application payload and protocol headers.
- **Delivery rate**: the percentage of successfully received messages relative to the total number of transmitted messages.
- **Energy consumption**: the energy required during message transmission and reception.

The average results reported by the study are summarized in Table 7.

Table 7: Experimental performance of MQTT, CoAP, and HTTP reported by Khaleefah et al.

Protocol	**Latency (ms)**	**Overhead (bytes)**	**Delivery Rate (%)**	**Energy (mWh)**
MQTT	22	168	99.7	0.49
CoAP	18	140	99.3	0.42
HTTP	87	610	96.1	1.12

Under the experimental conditions of the study, CoAP achieved the lowest latency, communication overhead, and energy consumption. MQTT achieved the highest delivery rate, while HTTP showed higher latency, message overhead, and energy consumption than MQTT and CoAP. The authors also performed statistical analysis using one-way ANOVA and Tukey's HSD test and reported statistically significant differences among the protocols for the evaluated metrics [2].

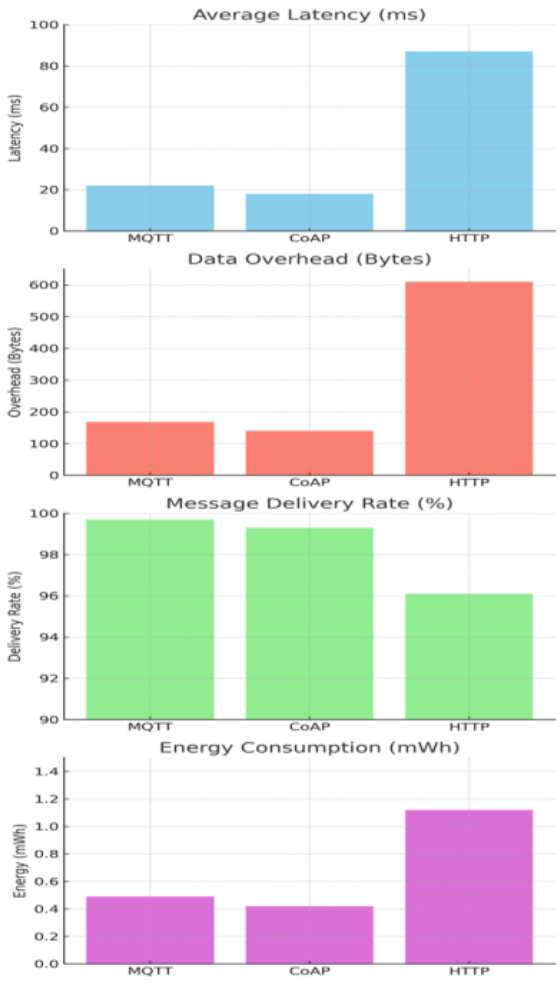


Figure 11: Comparison of MQTT, CoAP, and HTTP in terms of latency, message overhead, delivery rate, and energy consumption.

Figure 11 shows different trade-offs among the three protocols. CoAP provides the lowest latency, overhead, and energy consumption, while MQTT achieves the highest delivery reliability through its publish/subscribe model and QoS support. HTTP has higher communication cost in constrained environments but remains useful for integration with conventional web services. Therefore, protocol selection should be based on application requirements rather than assuming a single universally best protocol.

Based on the observed trade-offs, the authors proposed a protocol-selection flowchart that relates protocol choice to two main considerations: device resource constraints and message-delivery reliability requirements.

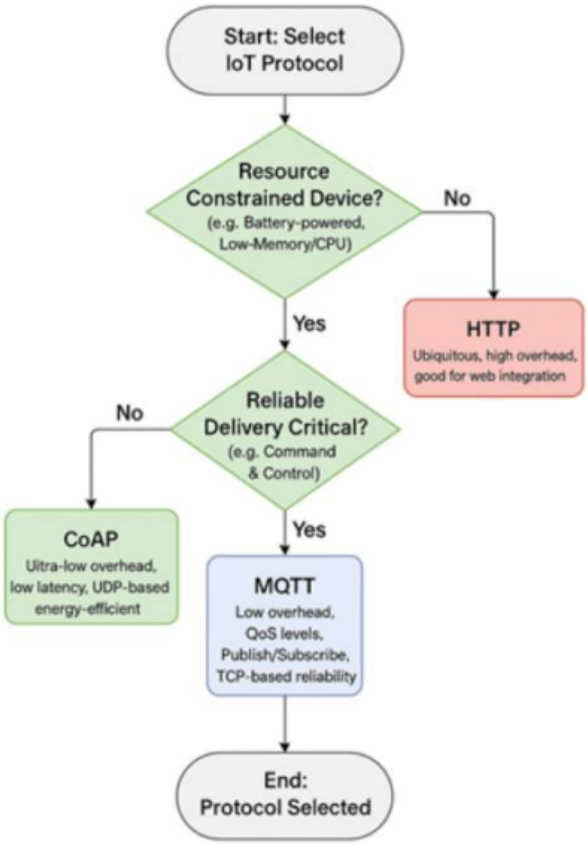


Figure 12: Protocol selection framework based on device constraints and reliability requirements.

As shown in the figure, the framework selects protocols according to device constraints and reliability requirements. CoAP is suitable when low latency, overhead, and energy consumption are prioritized, MQTT when reliable delivery is more important, and HTTP when integration with conventional web services is needed. For the proposed PPE system, this comparison is useful because the communication layer is still under evaluation, and the final choice should balance latency, reliability, resource efficiency, network stability, and backend compatibility.

Consequently, the results of this study are used as a reference rather than as a final protocol-selection decision. The three protocols should still be evaluated according to the actual workload and network conditions of the PPE monitoring system before the communication architecture is finalized.

Research 3

- **Title:** Performance Evaluation of MQTT and CoAP via a Common Middleware

- **Author:**Dinesh Thangavel, Xiaoping Ma, Alvin Valera, Hwee-Xian Tan and Colin Keng-Yan TAN
- **Year:** 2014
- **Link:** https://www.researchgate.net/publication/267636202$_Performance_evaluation_of_MQTT_and_CO$
- **Summary content:**

Thangavel et al. investigated the performance of MQTT and CoAP for gateway-to-backend communication in resource-constrained IoT environments. The study proposed a common middleware that allows different communication protocols to be accessed through a unified programming interface. Using this middleware, the authors compared MQTT and CoAP under different network conditions, focusing mainly on end-to-end delay and bandwidth consumption.

MQTT is implemented using a topic-based publish/subscribe model over TCP and supports three Quality of Service (QoS) levels. CoAP, in contrast, is based on REST-style communication over UDP and provides confirmable and non-confirmable messages for reliability. These differences make MQTT and CoAP suitable for different communication requirements and network conditions.

The common middleware was designed to be extensible, to expose a common API, and to support future adaptive protocol selection. Functions such as `publishMessage()` and `isPublishSuccess()` allow the upper application layer to interact with either MQTT or CoAP without directly handling protocol-specific details. For MQTT, messages are mapped to broker topics, while for CoAP, the middleware generates the corresponding URI.

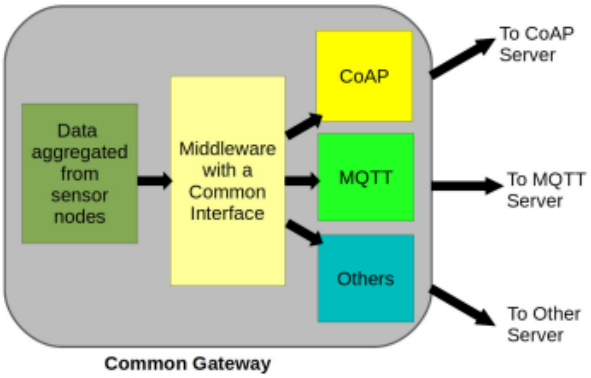


Figure 13: Common middleware architecture for MQTT and CoAP.

The experimental setup used a BeagleBoard-xM as the gateway, a laptop running the MQTT and CoAP servers, and a WAN emulator to control the packet loss rate. Message delay and

total transmitted bytes were recorded to evaluate protocol performance under different network conditions.

The results showed that MQTT achieved lower delay than CoAP when packet loss was low. However, as packet loss increased, CoAP achieved lower delay because MQTT suffered from additional TCP retransmission overhead. In addition, CoAP generally produced lower overhead for small messages, while the relative performance changed as message size and packet loss increased.

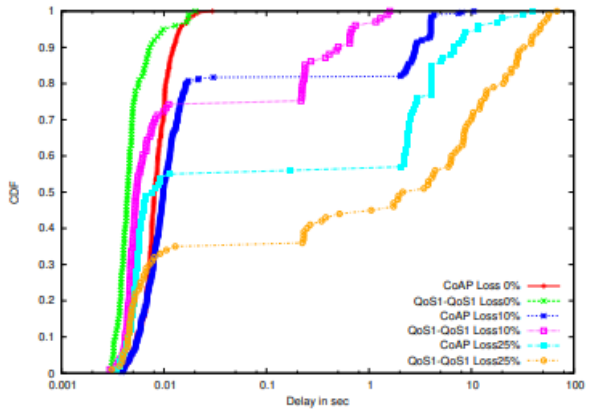


Figure 14: Influence of packet loss on MQTT and CoAP message delay.

The study concludes that protocol performance depends strongly on network conditions and application requirements. Therefore, MQTT and CoAP should not be considered universally superior to each other. The proposed middleware provides a basis for future adaptive protocol selection, where the gateway could choose the most suitable protocol according to current network conditions and optimization goals such as latency or bandwidth usage.

3.3 Comparison

These studies provide complementary perspectives on REST HTTP, MQTT, and CoAP, especially regarding unstable network conditions, communication efficiency, delivery reliability, and protocol selection.

Table 8: Main contributions of the reviewed studies to the proposed PPE monitoring system.

Study	**Main Contribution**	**Relevance to the PPE System**
Study 1 [1]	REST HTTP buffering and retransmission under unstable network conditions.	Shows that HTTP requires careful handling of buffering, timeout, reconnection, and retransmission when construction-site connectivity is unstable. It also supports separating communication failures from the local AI inference pipeline.
Study 2 [2]	Direct empirical comparison of MQTT, CoAP, and HTTP based on latency, overhead, delivery rate, and energy consumption.	Provides practical metrics for comparing the three candidate protocols and highlights the trade-off between communication efficiency, reliability, and backend compatibility.
Study 3 [3]	MQTT and CoAP comparison under different packet-loss rates and message sizes using a common middleware.	Shows that protocol performance changes with network conditions and supports the use of a protocol-independent communication interface for evaluation.

3.3.1 Experimental Evidence from the Reviewed Studies

In addition to the qualitative characteristics of each protocol, the reviewed studies provide experimental evidence regarding latency, delivery reliability, communication overhead, packet-loss behavior, and buffering. These results provide a more practical basis for evaluating the candidate protocols for the proposed PPE monitoring system.

Table 9: Consolidated experimental evidence for HTTP, MQTT, and CoAP.

Protocol	**Evidence from Study 1**	**Evidence from Study 2**	**Evidence from Study 3**
HTTP	199-byte JSON messages, 3-s generation interval, and 15-s timeout were evaluated under lossy connectivity. Larger buffers did not always improve performance, indicating the need for careful buffering, retry, and reconnection design.	87 ms latency, 610-byte overhead, 96.1% delivery rate, and 1.12 mWh energy consumption. HTTP showed the highest latency and overhead among the three protocols.	Not evaluated.
MQTT	Not evaluated.	22 ms latency, 168-byte overhead, 99.7% delivery rate, and 0.49 mWh energy consumption. MQTT achieved the highest delivery rate in the testbed.	Lower delay than CoAP at low packet-loss rates. Both protocols achieved 100% delivery in the reported simple testbed. TCP retransmission increased delay as packet loss increased.
CoAP	Not evaluated.	18 ms latency, 140-byte overhead, 99.3% delivery rate, and 0.42 mWh energy consumption. CoAP achieved the lowest latency and overhead.	Lower delay than MQTT as packet loss increased. For small messages, CoAP generally generated lower additional traffic under low-to-moderate packet loss.

Table 10: Comparison of candidate protocols according to the PPE system requirements.

Requirement	**HTTP**	**MQTT**	**CoAP**
Reliable transmission under unstable networks	Requires explicit buffering, retry, timeout, and reconnection mechanisms.	Strong delivery support through QoS, but TCP retransmission may increase delay under high packet loss.	Confirmable messages provide reliability and can achieve lower delay as packet loss increases.
Timely violation updates	Highest latency in the direct comparison (87 ms).	Low latency (22 ms) with the highest delivery rate (99.7%).	Lowest latency (18 ms) and low overhead, with 99.3% delivery rate.
Communication efficiency	Highest overhead (610 bytes).	Moderate overhead (168 bytes).	Lowest overhead (140 bytes).
Suitability for PPE event alerts	Simple backend integration, but less efficient for constrained communication.	Well suited for reliable event-driven alerts.	Suitable when low latency and low overhead are prioritized.

Table 11: Advantages and limitations of candidate communication protocols.

Protocol	**Advantages**	**Limitations / Considerations**
HTTP	Simple request–response model; straightforward integration with REST-based backend services; easy to implement and debug.	Higher communication overhead; requires careful handling of buffering, timeout, retransmission, and reconnection under unstable network conditions.
MQTT	Publish/subscribe architecture; configurable QoS levels; strong support for reliable event delivery; suitable for event-driven communication and multiple subscribers.	Requires an MQTT broker; higher QoS levels increase communication overhead; TCP retransmission may increase delay under high packet loss.
CoAP	Low latency and communication overhead; lightweight REST-oriented communication; suitable for constrained devices and networks.	Reliability mechanisms are less flexible than MQTT QoS; UDP retransmission may become inefficient for larger messages under high packet loss; backend integration may require additional support.

3.3.2 Implications for the Proposed PPE Monitoring System

The reviewed studies show that no protocol is universally superior. Each protocol provides a different trade-off depending on the application requirements and network conditions.

For the proposed PPE monitoring system, reliable and low-latency transmission of violation events is a primary requirement. In the direct three-protocol experiment, MQTT achieved a 99.7% delivery rate with an average latency of 22 ms, while CoAP achieved slightly lower latency (18 ms) and overhead but a slightly lower delivery rate (99.3%). HTTP showed considerably higher latency and overhead under the same experimental conditions [2].

These results make MQTT a strong candidate when delivery reliability is prioritized, while CoAP remains an important alternative when latency and communication efficiency are more critical. HTTP remains relevant mainly because of its straightforward integration with conventional REST-based backend services.

However, the reviewed studies also indicate that protocol performance changes under different network conditions. In particular, packet loss can significantly affect delay and retransmission behavior. Therefore, HTTP, MQTT, and CoAP should still be evaluated under the same PPE workload before the final protocol is selected.

The evaluation should consider latency, delivery success rate, communication overhead, packet-loss behavior, reconnection performance, and implementation complexity. In addition, the project should measure AI inference performance during network failures to verify that communication problems do not interrupt the local PPE detection process.

3.3.3 Current Selection Direction

Based on the current communication requirements and the experimental evidence reported in the reviewed studies, MQTT currently appears to be a strong candidate for the proposed PPE monitoring system.

The main reason is that the system prioritizes reliable delivery of violation events, while MQTT provides configurable QoS mechanisms and a publish/subscribe communication model that is well suited to event-driven alerts. However, CoAP remains an important lightweight alternative because of its low latency and communication overhead, while HTTP is still relevant as a baseline due to its straightforward integration with conventional REST-based backend services.

Nevertheless, the reviewed studies do not fully address all requirements of the proposed PPE monitoring system, particularly whether communication failures affect the performance of the local Edge AI inference pipeline. Therefore, the group plans to evaluate the system under stable connectivity, packet loss, and temporary network disconnection while measuring both communication performance and AI inference FPS/latency. This experiment is intended to verify that network failures can be handled independently without significantly interrupting

the local PPE detection process.

For this reason, the communication layer should remain protocol-independent at the current stage. Although MQTT currently appears promising, the group does not intend to commit to it prematurely. HTTP, MQTT, and CoAP will be evaluated under the same PPE workload and network conditions to ensure a fair comparison.

The final protocol will be selected based on measured results, including delivery success rate, latency, communication overhead, reconnection behavior, and the impact on local AI inference performance. If MQTT demonstrates reliable delivery with acceptable latency and overhead while maintaining stable PPE detection performance, it may be selected as the primary communication protocol. However, if another candidate protocol provides a better overall trade-off under the actual workload and deployment conditions, the group will reconsider the communication design and adopt the more suitable option.

4 Problem Definition and Deployment Requirements

4.1 Project Context

This project investigates a real-time Personal Protective Equipment (PPE) monitoring system intended for construction and industrial environments. The detection task focuses primarily on identifying hard hats and safety vests from camera images. The optimization part of the project focuses on transforming the trained detector into a model that can execute efficiently on resource-constrained edge hardware while maintaining acceptable detection accuracy.

The main responsibilities considered in this part are model compression, model conversion, selection of an inference runtime, hardware compatibility, and deployment benchmarking. Model architecture design and training are handled separately by the training component of the project.

4.2 Deployment Challenges

Deploying deep neural networks on edge devices is difficult because available memory, computing capability, and energy are considerably more restricted than on desktop or cloud systems. Therefore, model accuracy alone is insufficient when evaluating a deployment.

The project must consider model storage, runtime memory, inference latency, hardware compatibility, and detection accuracy. In addition, model execution time should be distinguished from end-to-end latency, which may also include image preprocessing and detection post-processing. Cordova-Cardenas et al. emphasize evaluating both model-level and system-level performance when designing an Edge AI deployment.

4.3 Evaluation Criteria

4.4 Research Questions

1. Which model optimization approach is suitable for the project requirements?
2. Which model representation and inference runtime support the selected approach?
3. Which hardware configuration should be investigated?
4. What experiments are required before confirming the final deployment configuration?

Table 12: Proposed deployment evaluation criteria.

Criterion	Measurement scope	Current target
Detection accuracy	mAP50, mAP50–95, and per-class performance for hard hats and safety vests, measured on the same evaluation dataset.	Establish from the FP32 baseline
Latency	Model-only inference time and complete end-to-end processing time, reported in milliseconds per frame.	Set after baseline measurement
Model storage	Size of the exported model file measured in MB.	Report baseline and optimized size
Runtime memory	RAM consumption while repeatedly executing inference on the target device.	Set after baseline measurement
CPU usage	Average process-level CPU utilization during a fixed inference workload.	Set after baseline measurement
Compatibility	Successful model conversion and execution on the selected runtime without unsupported operators or execution errors.	Successful execution
Implementation effort	Additional calibration, retraining, model conversion, or hardware-specific configuration required by the optimization method.	Prefer low to moderate effort

5 Review of Related Research

Together, these papers provide theoretical, deployment-oriented, and experimental perspectives relevant to the project.

5.1 Edge AI Background

Wang et al. identify the mismatch between increasingly complex neural networks and the limited computational and storage resources available on edge devices. The reviewed optimization methods include model sparsity, quantization, and knowledge distillation.

Quantization reduces the numerical precision used to represent model parameters and can substantially reduce model storage and computation. Knowledge distillation instead transfers information from a larger teacher network to a smaller student network. These techniques provide possible solutions for the PPE detector, although their effectiveness must be measured using the actual model and target hardware.

5.2 Optimization and Deployment Workflow

Cordova-Cardenas et al. propose an iterative deployment methodology consisting of requirement definition, model selection, optimization, deployment ecosystem selection, and final benchmarking. This approach is particularly relevant to the project because optimization cannot be selected independently from the target hardware and runtime.

Their evaluation framework also considers latency, throughput, memory usage, power consumption, and task-specific accuracy. Therefore, the optimization method selected for this project should be confirmed experimentally rather than only through theoretical compression ratios.

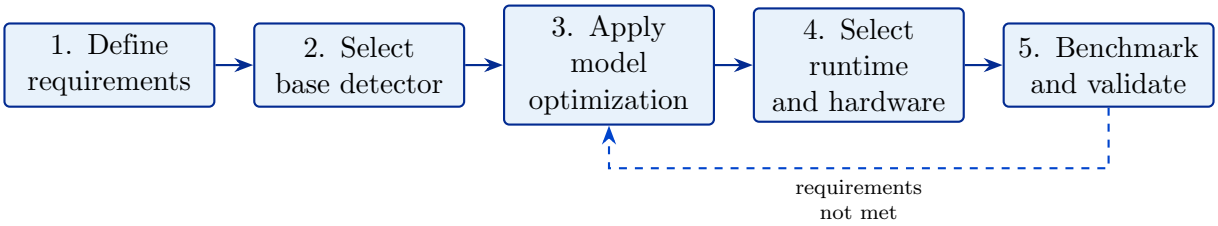


Figure 15: Adapted Edge AI deployment workflow for the proposed PPE system.

5.3 Experimental Compression Study

Han et al. introduce the Deep Compression pipeline consisting of pruning, trained quantization with weight sharing, and Huffman coding. Pruning removes low-importance connections

and retraining is then used to recover model performance. Weight sharing further reduces storage by representing multiple connections using shared values.

Their experiments reduced AlexNet from 240 MB to 6.9 MB and VGG-16 from 552 MB to 11.3 MB while maintaining the reported classification accuracy. However, these results were obtained on classification networks and cannot be assumed to apply directly to the PPE object detector.

5.4 Cross-Paper Synthesis and Research Gaps

The three papers consistently show that Edge AI deployment requires balancing accuracy and resource efficiency. Wang et al. provide the broad optimization options, Cordova-Cardenas et al. provide a deployment methodology, and Han et al. demonstrate that large reductions in storage are experimentally possible.

However, none of the reviewed experiments directly evaluates the detector, dataset, runtime, and hardware configuration used in this project. Therefore, the final optimization choice still requires project-specific experiments.

Table 13: Evidence extraction and comparison across the reviewed papers.

Study	Scope	Main evidence	Limitation	Application to this project
Wang et al. (2025)	Edge devices and AI applications across heterogeneous platforms.	Reviews sparsity, quantization, knowledge distillation, and architecture search using latency, memory, scalability, and deployment efficiency as key concerns.	Combines different models, tasks, and hardware; the findings do not predict PPE detector performance directly.	Defines the main optimization techniques and evaluation dimensions considered in this project.
Cordova-Cardenas et al. (2025)	MCUs, CPUs, GPUs, NPUs, Edge TPUs, and embedded platforms.	Proposes a five-stage workflow linking requirements, model selection, optimization, platform selection, and final benchmarking.	Aggregated evidence from heterogeneous systems limits direct transfer of individual benchmark values.	Provides the process used to organize deployment decisions and validation.
Han et al. (2016)	LeNet, AlexNet, and VGG-16 on CPU, desktop GPU, and mobile GPU platforms.	Combines pruning, trained quantization, weight sharing, and Huffman coding; reports 35–49× storage compression on the evaluated networks.	Uses older classification networks and mainly reports sparse-layer acceleration rather than a complete object-detection pipeline.	Shows that combined compression can be effective, while confirming the need for project-specific detector benchmarks.

6 Comparison of Model Optimization Methods

6.1 Baseline and Comparison Approach

The unoptimized FP32 detector will be used as the baseline. All optimized versions should be evaluated using the same input resolution, dataset split, batch size, hardware, preprocessing procedure, and runtime whenever possible. This controlled comparison prevents improvements caused by unrelated experimental differences from being incorrectly attributed to optimization.

6.2 Pruning

6.2.1 Principle and Procedure

Pruning removes parameters or structures considered to have limited contribution to the network. Han et al. prune small-weight connections and subsequently retrain the remaining network to recover performance. Their experiments reduced the number of connections substantially before the quantization stage.

6.2.2 Structured and Unstructured Approaches

Unstructured pruning removes individual weights and therefore creates sparse weight matrices. Structured pruning instead removes complete structures such as channels or filters. The practical acceleration achieved by either method depends on whether the selected inference runtime and hardware can efficiently execute the resulting representation.

6.2.3 Evidence, Limitations, and Project Suitability

Han et al. demonstrate strong storage reduction from pruning, but their implementation relies on sparse representations and older CNN architectures. Therefore, reduced parameter count does not automatically guarantee reduced latency for the PPE detector. Actual runtime support must first be verified.

6.3 Quantization

6.3.1 Numerical Precision and Quantization Principle

Quantization represents model parameters and, in some implementations, activations using lower numerical precision. Wang et al. identify INT8 quantization as an important technique for reducing memory and computation in edge deployment. FP16 conversion should be distinguished from integer quantization because FP16 remains a floating-point representation.

6.3.2 Post-Training Quantization and Quantization-Aware Training

Post-training quantization modifies an already trained model and therefore requires comparatively little additional training effort. Quantization-aware training introduces quantization effects during training and can better adapt the network to reduced precision, but requires additional training.

The trained quantization used by Han et al. is different from conventional INT8 PTQ. Their method clusters weights into shared values and retrains the cluster centroids.

6.3.3 Static and Dynamic Quantization

Static and dynamic quantization differ in when activation quantization parameters are determined. Their exact applicability depends on the selected runtime and the operators in the exported detector.

6.3.4 Calibration and Detection Accuracy

If INT8 post-training quantization is selected, representative calibration images should include different lighting conditions, object scales, occlusions, and distant PPE objects. Detection accuracy should then be compared using mAP50, mAP50–95, and relevant per-class results.

6.3.5 Evidence, Limitations, and Project Suitability

The reviewed literature indicates that quantization is one of the most practical methods for reducing Edge AI resource requirements. For this project, it is therefore a strong candidate for initial investigation. However, actual speed improvement and accuracy degradation must be measured experimentally.

6.4 Knowledge Distillation

6.4.1 Teacher–Student Approach

Knowledge distillation transfers information from a larger teacher model to a smaller student model. Instead of learning exclusively from ground-truth labels, the student also learns information generated by the teacher.

6.4.2 Training Requirements and Project Suitability

Unlike post-training quantization, knowledge distillation requires another training process and an appropriate teacher–student configuration. It may produce a naturally smaller de-

ployment model, but introduces additional training effort. For the current project schedule, it can therefore be considered as a secondary option if simpler post-training optimization is insufficient.

6.5 Combined Optimization Methods

Han et al. demonstrate that pruning, trained quantization, and Huffman coding can be applied sequentially. Their complete pipeline achieved reported storage reductions of 35–49× on the tested networks. Nevertheless, this should be treated as evidence that optimization methods can complement one another rather than as an expected compression ratio for the PPE model.

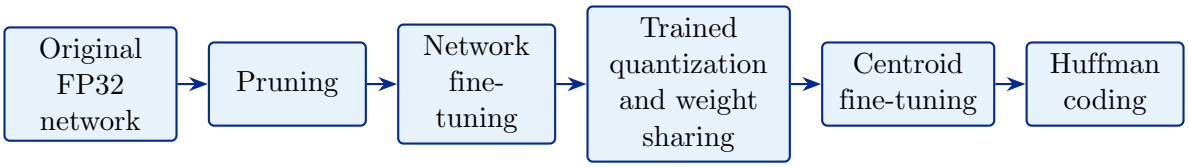


Figure 16: Deep Compression pipeline described by Han et al.

6.6 Comparative Assessment and Provisional Preference

Based on the reviewed literature, post-training quantization is a suitable method to investigate first because it directly targets model size and inference efficiency while requiring less additional training than distillation. Pruning may subsequently be investigated if the selected runtime provides efficient sparse execution. Knowledge distillation remains a possible alternative when retraining resources are available.

Table 14: Comparison of candidate model optimization approaches.

Method	Required data	Training effort	Deployment effect	Main risk	Project role
Pruning	Training data for post-pruning fine-tuning.	Moderate.	Reduces stored parameters; unstructured pruning creates sparse weights.	Sparse models may not run faster without runtime and hardware support.	Secondary option after sparse-execution support is verified.
Quantization	Representative calibration data for PTQ; training data for QAT.	Low for PTQ; moderate to high for QAT.	Reduces weight and activation precision, storage, and potentially latency.	Low-bit conversion may reduce detection accuracy or expose unsupported operators.	Preferred first experiment because PTQ has comparatively low implementation effort.
Knowledge distillation	Training data plus teacher outputs or features.	High.	Produces a smaller student model without requiring a sparse representation.	Results depend on teacher quality, student capacity, and loss design.	Alternative when retraining resources and a suitable teacher are available.
Combined methods	Data required by every selected stage.	High.	Can achieve greater compression by combining complementary techniques.	Complexity and accuracy loss can accumulate across stages.	Evaluate only after individual methods have been benchmarked separately.

7 Deployment Tools, ONNX, and Computation Graphs

7.1 From Trained Model to Deployment

The proposed deployment process begins with the trained model produced by the training stage. The model is exported into a deployment representation, checked for output consistency, optimized when appropriate, and executed using the selected inference runtime. The final model is then benchmarked under controlled conditions.

This follows the iterative deployment philosophy proposed by Cordova-Cardenas et al., where optimization, hardware selection, and validation are considered connected decisions rather than independent stages.

7.2 Training Frameworks

7.2.1 PyTorch

PyTorch is relevant because the trained detector can be developed and exported from the training environment before deployment. In this project, the optimization stage does not depend on PyTorch being the final inference runtime.

7.2.2 TensorFlow

TensorFlow provides an alternative model-development ecosystem and is closely associated with TensorFlow Lite for embedded inference. However, using it would introduce an additional conversion path if the original detector is developed in PyTorch.

7.3 Candidate Inference Runtimes

7.3.1 ONNX Runtime

Cordova-Cardenas et al. describe ONNX Runtime as an inference engine for models represented using the ONNX format and emphasize ONNX interoperability between deep-learning frameworks. This makes it a useful candidate for initial CPU-based experiments.

7.3.2 NVIDIA TensorRT

TensorRT is relevant if an NVIDIA GPU or Jetson platform is selected because it is intended for optimized inference on NVIDIA hardware.

7.3.3 TensorFlow Lite

TensorFlow Lite targets mobile and embedded inference and supports model optimization for constrained devices. Its suitability for this project depends on the conversion path from the selected detector and target hardware.

7.4 ONNX as an Intermediate Model Representation

ONNX should be distinguished from ONNX Runtime. ONNX is the model representation, whereas ONNX Runtime is an engine capable of executing ONNX models. An exported ONNX model represents inference as a computation graph containing operators and tensors connected between model inputs and outputs.

This representation allows the deployment stage to inspect the model independently from the original training code.

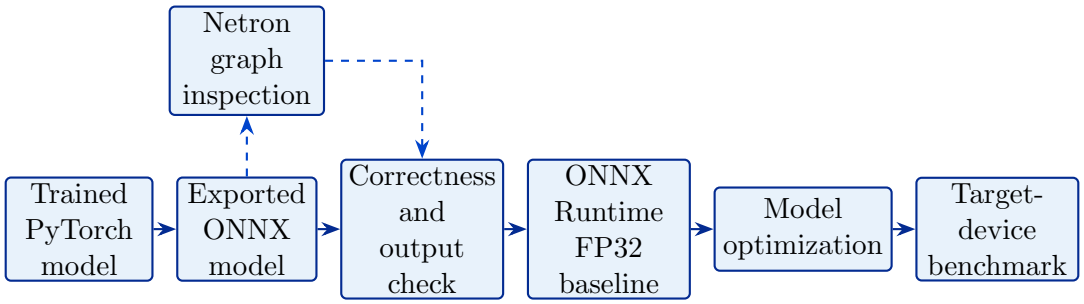


Figure 17: Proposed model deployment and validation workflow.

7.5 Computation Graph Inspection Using Netron

7.5.1 Inspection Procedure

After the final trained model is exported to ONNX, it will be opened using Netron. The inspection will record the model input and output tensors, tensor shapes, data types, principal operators, and connections between major network components.

7.5.2 Graph Observations and Potential Compatibility Issues

No final graph observations are reported at this stage because the final exported model has not yet been inspected. The purpose of the inspection will be to identify unexpected operators, output structures, or graph components that could cause deployment compatibility problems.

7.6 Tool Comparison and Proposed Workflow

The proposed workflow establishes ONNX Runtime as the initial FP32 baseline. TensorRT or TensorFlow Lite can later replace the inference stage if the selected hardware makes either runtime more appropriate.

Table 15: Comparison of candidate deployment tools.

Tool	Target platform	Model preparation	Main constraint	Project role
ONNX Runtime	General-purpose CPUs and supported accelerators through execution providers.	Export the trained detector to ONNX and verify operators, shapes, and outputs.	Compatibility depends on the exported graph and selected execution provider.	Initial FP32 baseline and first platform for controlled optimization tests.
TensorRT	NVIDIA GPUs and Jetson platforms.	Convert or compile the model for the installed TensorRT version and target GPU.	Operator and precision support are hardware- and version-dependent.	Preferred accelerated path if an NVIDIA Jetson is selected.
TensorFlow Lite	Mobile and embedded CPU or accelerator platforms.	Convert the detector into a TensorFlow Lite compatible representation.	The PyTorch-to-TFLite conversion path and model operators must be validated.	Alternative lightweight runtime for non-NVIDIA embedded deployment.

8 Hardware and Deployment Configuration

8.1 Device comparision

Table 16: Comparison framework for candidate deployment devices.

Candidate	Compute resources	Memory	Runtime path	Expected role	Required validation
NVIDIA Jetson (exact model TBD)	Embedded CPU and NVIDIA GPU; exact configuration to be recorded.	TBD	ONNX Runtime or TensorRT, depending on model compatibility.	Candidate for GPU-accelerated edge inference.	Availability, power mode, supported precision, FPS, latency, and memory use.
Raspberry Pi (exact model and accelerator TBD)	ARM CPU with an optional external accelerator if selected.	TBD	ONNX Runtime or TensorFlow Lite; accelerator runtime if applicable.	Candidate for lower-cost embedded CPU deployment.	Conversion support, sustained FPS, thermal behavior, and accelerator benefit.
Development PC (configuration to be recorded)	Available desktop CPU and GPU.	System RAM TBD	PyTorch and ONNX Runtime; TensorRT if a compatible NVIDIA GPU is available.	Reference platform for export checks and baseline measurements.	Record full configuration and separate desktop results from edge-device results.

8.2 Interpretation of Published Benchmarks

Published benchmark values cannot be directly transferred to the proposed system unless the model, hardware, numerical precision, input size, and measurement procedure are comparable. For example, Han et al. report layer-level acceleration for their compressed networks, but their models and execution environment differ substantially from the PPE detector.

Consequently, published benchmarks will be used as motivation rather than as predicted project performance.

8.3 Proposed Deployment Configuration

At the present stage, a reasonable provisional workflow is to export the trained detector to ONNX and establish an FP32 baseline using ONNX Runtime. Post-training quantization can then be evaluated under the same experimental conditions. The final edge device should remain provisional until runtime compatibility and hardware availability have been confirmed.

Table 17: Provisional deployment configuration and remaining validation.

Component	Provisional choice	Remaining validation
Detector	YOLO11n validation baseline	Confirm test-set accuracy and target-device inference performance.
Optimization	INT8 post-training quantization	Measure calibration sensitivity, accuracy loss, size reduction, and latency change.
Model representation	ONNX	Inspect the graph and verify input, output, operator, and shape compatibility.
Inference runtime	ONNX Runtime FP32 baseline	Confirm execution-provider support and compare against hardware-specific runtimes.
Execution device	To be selected after hardware benchmarking	Compare availability, FPS, latency, RAM, CPU/GPU use, and implementation effort.

8.4 Planned Validation

8.4.1 Experimental Controls

All measurements will use a fixed software environment, device configuration, input resolution, batch size, dataset split, and preprocessing procedure. Several warm-up inferences will be performed before timing so that initialization overhead does not distort the measurements.

8.4.2 Measurements and Reporting

Model size will be measured from the exported model file. Model-only latency and end-to-end processing latency will be recorded separately. Runtime RAM and CPU utilization will be monitored during repeated inference, while detection accuracy will be evaluated using the same validation or test data for both the FP32 and optimized models.

The final comparison will report absolute values and the difference between the baseline and optimized configurations

9 Research Summary and Next Steps

9.1 Synthesis of Literature Findings

The reviewed literature shows that successful Edge AI deployment requires both model optimization and hardware-aware evaluation. Quantization, pruning, and knowledge distillation are commonly investigated approaches, while deployment decisions should be evaluated through real measurements of accuracy, latency, memory, and resource usage.

Han et al. demonstrate that substantial compression is technically possible, but their results also show why results from one architecture should not be directly generalized to another.

9.2 Provisional Selections and Justification

The current provisional approach is to use ONNX as the intermediate representation, ONNX Runtime for the first deployment baseline, and post-training quantization as the first optimization method to investigate. These choices minimize additional training effort while allowing direct comparison between the original and optimized detector.

They remain provisional until the final model and target hardware are available.

9.3 Comparison with Reviewed Studies

Unlike Han et al., this project evaluates object detection rather than image classification. Unlike the broad surveys of Wang et al. and Cordova-Cardenas et al., the project will produce measurements for one specific PPE detection workload and deployment configuration.

Therefore, the project contributes a small application-specific evaluation rather than attempting to reproduce the general conclusions of the reviewed literature.

9.4 Remaining Research and Planned Experiments

The remaining work includes selecting the exact edge hardware, exporting the final trained model to ONNX, inspecting its computation graph in Netron, checking runtime compatibility, preparing representative calibration data, performing post-training quantization, and benchmarking the FP32 and optimized models.

The final deployment decision will be made only after these measurements are available.

10 Revised System Architecture

10.1 Architecture Revision Rationale

The system architecture proposed in Week 1 established the main direction of the project as an Edge AI-based PPE monitoring system consisting of camera acquisition, local AI inference, communication with a backend server, and monitoring through a dashboard.

In Week 2, the architecture was further refined into an asynchronous processing pipeline. Camera acquisition, AI inference, and communication were separated through intermediate queues so that blocking network operations would not directly interrupt the local PPE detection process.

The investigations performed in Week 3 further refine this architecture. The detection model, model optimization method, inference runtime, target hardware, and communication protocol are now treated as related deployment decisions rather than independent selections. The current results suggest several provisional directions, including YOLO11n as the stronger validation baseline, post-training quantization as the first optimization method to investigate, ONNX as the intermediate model representation, and ONNX Runtime as the initial deployment baseline.

However, these choices are not considered final until they are validated on the target edge hardware. Similarly, MQTT currently appears to be a strong communication candidate, while CoAP and HTTP remain relevant alternatives under different communication requirements.

Therefore, the revised architecture preserves the asynchronous system structure introduced in Week 2 while keeping the deployment and communication components configurable for further experimental evaluation.

10.2 Revised End-to-End System Flow

The current end-to-end processing flow of the proposed PPE monitoring system is illustrated in Fig. 18.

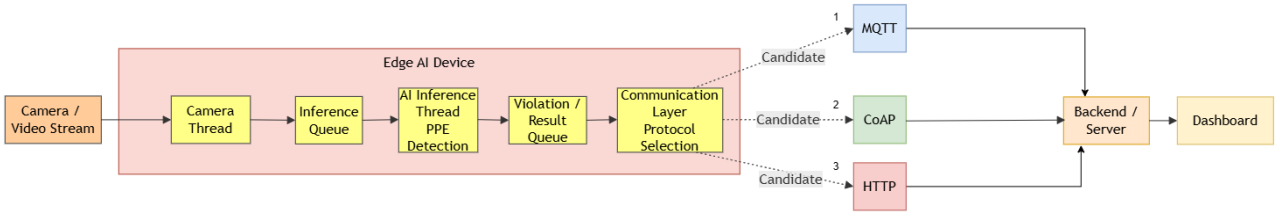


Figure 18: Revised end-to-end architecture of the proposed PPE monitoring system.

The processing pipeline begins with the Camera Thread, which continuously acquires frames

from the input video stream. Frames are transferred through the Inference Queue to the local Edge AI inference stage. PPE detection is performed locally so that the core monitoring function does not depend on continuous network availability.

When a relevant PPE violation is detected, the corresponding information is placed in the Violation / Result Queue instead of being transmitted directly from the inference thread. The communication component then processes the queued events independently.

This separation preserves the asynchronous design from Week 2 and allows communication failures, reconnection, buffering, and retransmission to be handled without directly blocking the local AI inference pipeline.

10.3 Current Provisional Configuration

Based on the findings obtained so far, the current provisional system configuration can be summarized as follows:

Table 18: Current provisional configuration of the proposed PPE monitoring system.

Component	**Current Direction**	**Remaining Validation**
Detection model	YOLO11n as the stronger validation baseline	Inference FPS, latency, and final test-set performance on target hardware
Model representation	ONNX	Operator compatibility and computation-graph inspection
Optimization	Post-training quantization as the first method to investigate	Accuracy degradation, latency improvement, model-size reduction
Inference runtime	ONNX Runtime as initial FP32 baseline	Comparison with TensorRT or TFLite depending on selected hardware
Edge hardware	Jetson, Raspberry Pi, and development PC remain candidates	Availability, runtime support, FPS, latency, RAM, and CPU/GPU utilization
Communication protocol	MQTT as a strong candidate; CoAP and HTTP remain alternatives	Latency, delivery rate, overhead, packet-loss behavior, and reconnection

11 Evaluation Plan

The evaluation plan aims to validate the provisional design decisions under controlled experimental conditions. The system will be evaluated at four levels: detection performance, edge deployment performance, communication performance, and complete system behavior.

11.1 Model Evaluation

The candidate detection models will be evaluated using precision, recall, mAP50, mAP50:95, class-specific AP, and representative missed or false detections. YOLO11n currently provides the stronger validation baseline, but final selection also requires deployment benchmarking on the target edge hardware.

11.2 Edge Deployment Evaluation

The deployment configuration will be evaluated using inference latency, FPS, model size, RAM usage, and CPU/GPU utilization. The current provisional workflow uses ONNX as the intermediate representation, ONNX Runtime as the initial FP32 baseline, and post-training quantization as the first optimization method to investigate.

All candidate configurations should be tested under comparable input size, batch size, preprocessing, and hardware conditions.

11.3 Communication Protocol Evaluation

HTTP, MQTT, and CoAP will be evaluated using the same violation-event workload. The main metrics include message latency, delivery success rate, communication overhead, packet-loss behavior, and reconnection performance.

The protocols will be tested under stable connectivity, controlled packet loss, and temporary disconnection followed by reconnection.

11.4 System-Level Evaluation

The complete system will also be tested to verify that communication failures do not significantly interrupt local PPE inference. AI FPS and inference latency will therefore be measured together with communication performance under different network conditions.

11.5 Experimental Workflow

The overall evaluation workflow is illustrated in Fig. 20.

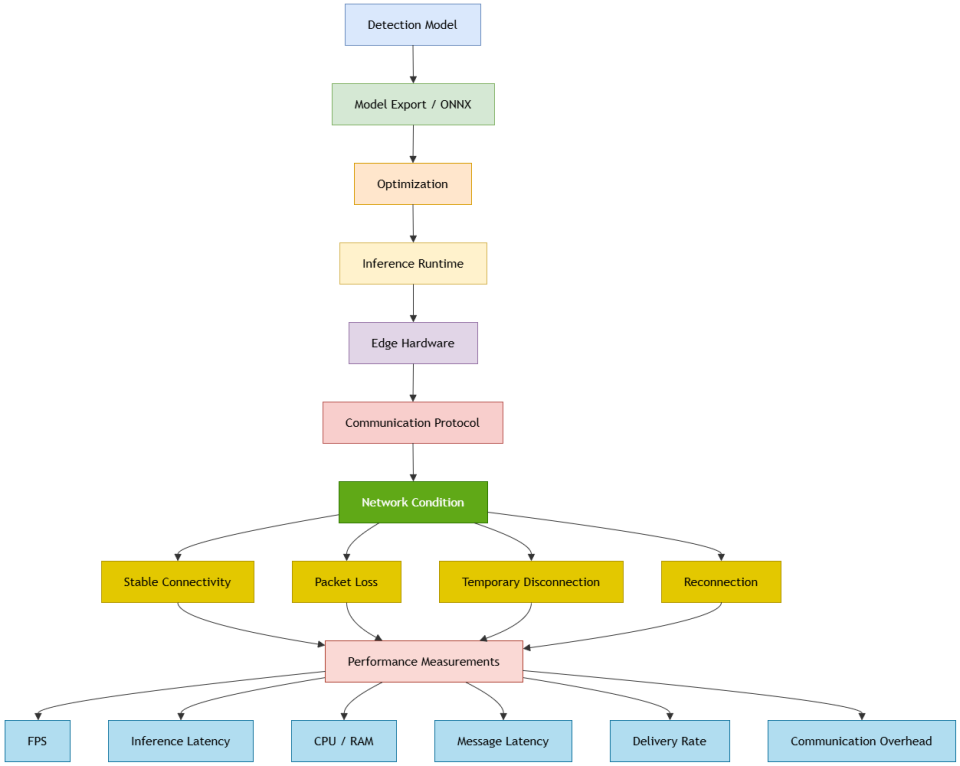


Figure 19: Proposed end-to-end experimental workflow for evaluating model, deployment, communication, and system-level performance.

12 Week 3 Findings and Next Steps

12.1 Current Findings

Week 3 refined several early design assumptions from Weeks 1 and 2. YOLO11n currently provides the stronger validation baseline, while ONNX, ONNX Runtime, and post-training quantization form the current provisional deployment direction. The final edge hardware has not yet been selected.

For communication, MQTT appears to be a strong candidate because of its reliable event-delivery mechanisms, while CoAP remains attractive for low-latency and low-overhead communication and HTTP remains useful for straightforward backend integration.

The asynchronous architecture is retained so that communication failures do not directly

interrupt local PPE inference.

12.2 Remaining Decisions

The final detection model, optimization method, inference runtime, edge hardware, communication protocol, and buffering/retransmission strategy still require experimental validation.

These decisions will be based on measured performance rather than the initial assumptions made in the earlier project stages.

12.3 Next Steps

The next stage will focus on:

- exporting and benchmarking the candidate model on the target edge hardware;
- evaluating the FP32 baseline and optimized model;
- benchmarking HTTP, MQTT, and CoAP under stable, lossy, and disconnected network conditions;
- measuring whether communication failures affect local AI inference;
- updating the architecture and selecting the final deployment configuration based on the experimental results.

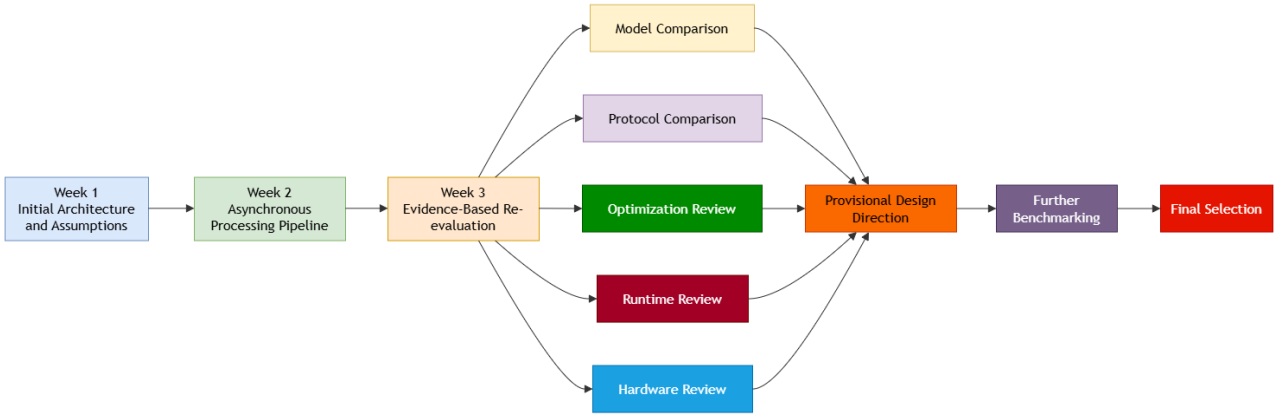


Figure 20: Evolution of the project from the initial design in Weeks 1–2 to the evidence-based re-evaluation and provisional decisions in Week 3.

References

[1] Cao Vien Phung, Mounir Bensalem and Admela Jukan, "Benchmarking Buffer Size in IoT Devices Deploying REST HTTP," [Online]. Available: https://arxiv.org/pdf/2101.02615, 2021.

[2] Rafid M. Khaleefah, Ali A. Abed, Mahmood A. Al-Shareeda, "Empirical Evaluation of MQTT, CoAP and HTTP for Smart City IoT Applications," [Online]. Available: https://www.researchgate.net/publication/395833265_Empirical_Evaluation_of_MQTT_CoAP_and_HTTP_for_Smart_City_IoT_Applications, 2025.

[3] Dinesh Thangavel, Xiaoping Ma, Alvin Valera, Hwee-Xian Tan and Colin Keng-Yan TAN, "Performance Evaluation of MQTT and CoAP via a Common Middleware," [Online]. Available: https://www.researchgate.net/publication/267636202_Performance_evaluation_of_MQTT_and_CoAP_via_a_common_middleware, 2014.

[4] T. Wang, J. Guo, B. Zhang, G. Yang and D. Li, "Deploying AI on Edge: Advancement and Challenges in Edge Intelligence," *Mathematics*, vol. 13, no. 11, Art. no. 1878, 2025. [Online]. Available: https://doi.org/10.3390/math13111878.

[5] R. Cordova-Cardenas, D. Amor and Á. Gutiérrez, "Edge AI in Practice: A Survey and Deployment Framework for Neural Networks on Embedded Systems," *Electronics*, vol. 14, no. 24, Art. no. 4877, 2025. [Online]. Available: https://doi.org/10.3390/electronics14244877.

[6] S. Han, H. Mao and W. J. Dally, "Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding," *arXiv*, 2016. [Online]. Available: https://doi.org/10.48550/arXiv.1510.00149.